DukaanPilot AI — Technical Architecture & System Report

Autonomous Closed-Loop AI Business Growth Worker for Micro-Retailers | Sneha Bhandar (Ranchi)

Target Retailer: Sneha Bhandar (Grocery / Kirana)	**Location:** Shiv Shakti Nagar, Booty More, Ranchi, Jharkhand (834009)
Milestone Goal: Baseline ■8,000 → Target ■10,000 / month	**Key Orchestrator:** LangGraph Cyclic StateGraph + SQLite Checkpointer
Speech Synthesis: Sarvam AI TTS (Bilingual Hindi / English)	**Safety Architecture:** Human-in-the-Loop Interrupt Gate + Zero Phantom Uplift

1. Executive Summary & Problem Formulation

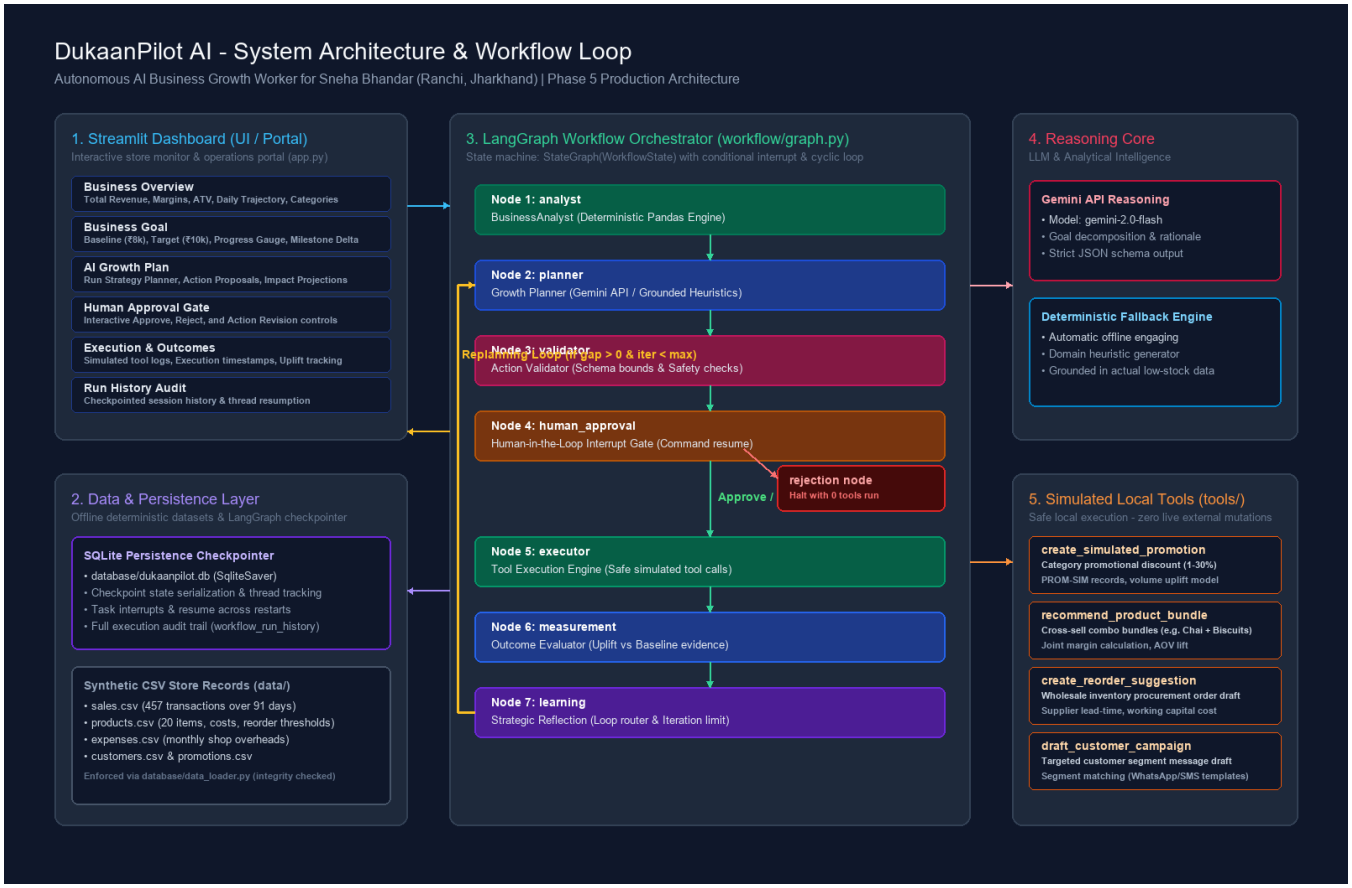
Micro-retailers across India—such as neighborhood *kirana* stores—operate on thin margins and juggle multiple operational responsibilities: purchasing inventory, balancing cash vs. credit (*khata*), and servicing walk-in patrons. Store owners lack dedicated data analysts or merchandisers, resulting in trapped working capital, frequent stockouts of fast-moving staples, and missed margin opportunities. **DukaanPilot AI** transforms passive retail ledger records into proactive, goal-directed interventions. Operating within a strictly verified offline simulation boundary for **Sneha Bhandar** (Ranchi), the autonomous worker ingests 91 days of transactions, formulates strategic promotions and bundle deals, halts at mandatory human approval gates, executes simulated operational tools, and evaluates uplift against verified historical baselines.

2. Why an Autonomous AI Worker Rather Than a Chatbot?

Architectural Dimension	Conventional Chatbot	DukaanPilot AI Worker
Operating Mode	Passive request-response; waits for queries.	Proactive goal-driven loop; drives monthly targets.
Arithmetic Precision	Delegates math to LLM (frequent hallucinations).	Deterministic Pandas engine; 100% verified math.
Action Execution	Emits markdown advice; cannot execute actions.	Dispatches schema-validated tools with structured logs.
State Durability	Ephemeral context; resets on browser refresh.	SQLite-backed checkpoints; resumes across restarts.
Safety Architecture	Vague system prompts ('please be careful').	Hard LangGraph interrupt() gates requiring authorization.
Closed-Loop Learning	Open-loop; zero outcome verification.	Measures uplift vs baselines and replans adaptively.

3. System Architecture & Component Design

DukaanPilot AI employs a modular 4-layer architecture comprising (1) Presentation (Streamlit + Sarvam AI UI), (2) Orchestration (LangGraph StateGraph), (3) Deterministic Analytics & LLM Reasoning Core (Pandas + Gemini 2.0 Flash), and (4) Persistence (SQLite checkpointer).


DukaanPilot AI - System Architecture & Workflow Loop
Autonomous AI Business Growth Worker for Sneha Bhandar (Ranchi, Jharkhand) | Phase 5 Production Architecture
1. Streamlit Dashboard (UI / Portal)
Interactive store monitor & operations portal (app.py)
Business Overview
Total Revenue, Margins, ATV, Daily Trajectory, Categories
Business Goal
Baseline (₹8k), Target (₹10k), Progress Gauge, Milestone Delta
AI Growth Plan
Run Strategy Planner, Action Proposals, Impact Projections
Human Approval Gate
Interactive Approve, Reject, and Action Revision controls
Execution & Outcomes
Simulated tool logs, Execution timestamps, Uplift tracking
Run History Audit
Checkpointed session history & thread resumption
2. Data & Persistence Layer
Offline deterministic datasets & LangGraph checkpointer
SQLite Persistence Checkpointer
• database/dukaanpilot.db (SqliteSaver)
• Checkpoint state serialization & thread tracking
• Task interrupts & resume across restarts
• Full execution audit trail (workflow_run_history)
Synthetic CSV Store Records (data/)
• sales.csv (457 transactions over 91 days)
• products.csv (20 items, costs, reorder thresholds)
• expenses.csv (monthly shop overheads)
• customers.csv & promotions.csv
Enforced via database/data_loader.py (integrity checked)
3. LangGraph Workflow Orchestrator (workflow/graph.py)
State machine: StateGraph(WorkflowState) with conditional interrupt & cyclic loop
Node 1: analyst
BusinessAnalyst (Deterministic Pandas Engine)
Node 2: planner
Growth Planner (Gemini API / Grounded Heuristics)
Node 3: validator
Action Validator (Schema bounds & Safety checks)
Replanning Loop (if gap > 0 & iter < max)
Node 4: human_approval
Human-in-the-Loop Interrupt Gate (Command resume)
rejection node
Halt with 0 tools run
Approve /
Node 5: executor
Tool Execution Engine (Safe simulated tool calls)
Node 6: measurement
Outcome Evaluator (Uplift vs Baseline evidence)
Node 7: learning
Strategic Reflection (Loop router & Iteration limit)
4. Reasoning Core
LLM & Analytical Intelligence
Gemini API Reasoning
• Model: gemini-2.0-flash
• Goal decomposition & rationale
• Strict JSON schema output
Deterministic Fallback Engine
• Automatic offline engaging
• Domain heuristic generator
• Grounded in actual low-stock data
5. Simulated Local Tools (tools/)
Safe local execution - zero live external mutations
create_simulated_promotion
Category promotional discount (1-30%)
PROM-SIM records, volume uplift model
recommend_product_bundle
Cross-sell combo bundles (e.g. Chai + Biscuits)
Joint margin calculation, AOV lift
create_reorder_suggestion
Wholesale inventory procurement order draft
Supplier lead-time, working capital cost
draft_customer_campaign
Targeted customer segment message draft
Segment matching (WhatsApp/SMS templates)

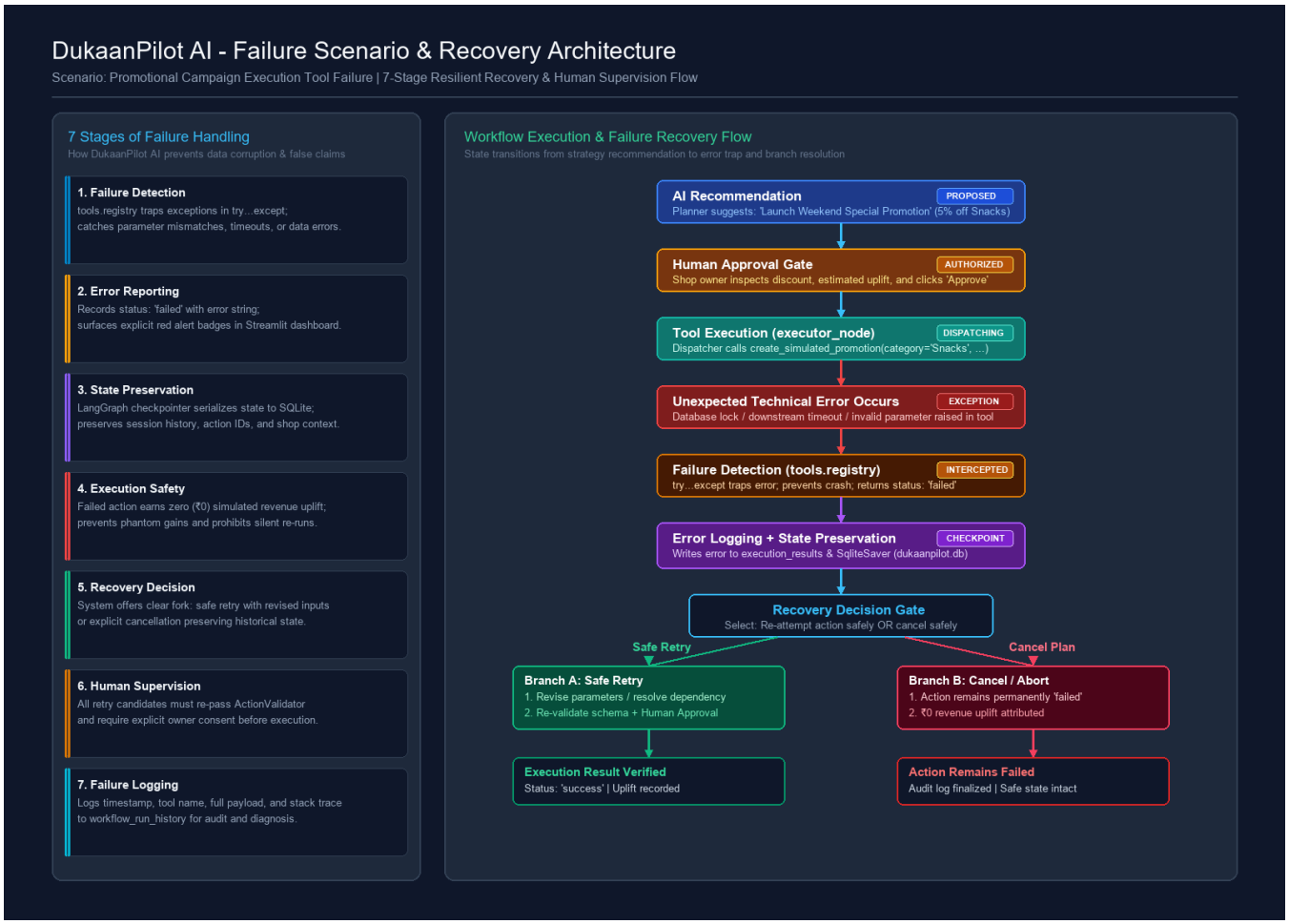
4. Dedicated Failure Scenario: Business Tool Execution Failure

Scenario Context: DukaanPilot AI analyzes 91 days of transaction ledgers for Sneha Bhandar and detects that packaged snacks and beverages have high footfall but small ticket sizes. The Growth Planner (planner_node) formulates a recommendation: launch a 'Weekend Family Snack Special' offering a 5% discount on snacks to boost basket value (projecting an estimated monthly uplift of +■350). The shopkeeper inspects the proposal in the dashboard and clicks **'Approve'**. However, when the worker attempts to dispatch create_simulated_promotion in executor_node, the simulated business tool encounters an **unexpected technical error** (such as a database file lock, downstream repository timeout, or catalog parameter serialization mismatch).

The 7 Stages of Resilient Failure Handling:

Stage 1: Failure Detection	The dispatcher (tools.registry.execute_action) wraps every tool call in a structured try...except Exception as exc: boundary. Unhandled exceptions are trapped immediately before they can crash the LangGraph workflow runtime, constructing an explicit payload: {'status': 'failed', 'tool': ..., 'error': str(exc)}.
Stage 2: Error Reporting	The system never masks errors or claims false success. The executor_node appends a structured failure entry into WorkflowState['execution_results'] with timestamp and full error diagnostics. The Streamlit dashboard renders prominent alert badges (■ FAILED) notifying the shopkeeper immediately.
Stage 3: State Preservation	LangGraph's SqliteSaver checkpointer commits the complete state to database/dukaanpilot.db after every transition. The shop's baseline metrics, original recommendations, approval timestamps, and partial tool logs remain intact. If the application restarts, execution resumes from the saved checkpoint without data loss.
Stage 4: Execution Safety	The failed action is strictly marked as failed and never marked completed. The evaluation engine (workflow/evaluator.py) filters successful runs only; failed actions contribute exactly ■**0.00** simulated revenue uplift. No phantom gains are attributed. Unaffected batch tools continue executing independently.
Stage 5: Recovery Strategy	The learning_node reflects on failures, setting should_replan = True. In the UI, the owner is offered two distinct paths: (A) **Safe Retry / Revision** via the revision panel with adjusted parameters, or (B) **Safe Cancellation**, terminating the run with zero unapproved side effects.
Stage 6: Human Supervision	No autonomous background retries alter retail state without human oversight. Any retried or modified action must re-pass ActionValidator (schema rules & ≤30% discount ceiling) and pause at the human_approval gate via interrupt() for explicit owner consent.
Stage 7: Failure Logging	Full diagnostic telemetry—action ID, tool name, submitted parameters, timestamp (executed_at), and error stack trace—is persisted to SQLite (workflow_run_history) for post-mortem auditing, compliance, and debugging.

5. Failure Recovery Flow Architecture



6. Failure Handling Evaluation Matrix

Failure Condition	Expected Worker Behaviour	Safety Mechanism	Recovery Approach
Tool Execution Failure *(Timeout, I/O lock, key mismatch)*	Intercept error at boundary; set status='failed'; log error; attribute ■0 uplift; continue other batch tools.	try...except in registry.py; zero-uplift filter in evaluator.py; schema boundary rules.	Isolate failed tool; trigger replanning in learning node; offer owner parameter revision & re-approval.
Missing / Corrupted Data *(Missing CSV, NaN price, broken foreign keys)*	Halt data ingestion before computation; prevent corrupt records from reaching LLM or tools; alert user.	validate_data_integrity() enforces primary keys, non-null values, and positive prices.	Report exact schema breach; fall back to verified default synthetic dataset; emit diagnostic logs.
Gemini API Unavailable *(Network timeout, quota exhausted, invalid key)*	Formulate grounded strategic actions without crashing, hanging, or requiring active internet.	Heuristic rule-based fallback in generate_fallback_plan(); structured JSON validation.	Automatically route to deterministic recommendations; tag plan with planner_source: 'fallback'.
Interrupted Workflow *(Tab close, server kill, power outage mid-session)*	Retain exact state snapshot and pending interrupts; avoid duplicate runs or restarting analysis.	LangGraph SqliteSaver checkpointer commits state to SQLite after every node execution.	Re-open session with identical thread ID; get_state() restores pending interrupt cleanly.
Duplicate Execution Attempt *(Double click, browser refresh, multi-POST)*	Execute approved actions exactly once; ignore redundant clicks; prevent multiple orders/discounts.	Idempotent thread IDs; state execution tracking (workflow_status: 'executed').	Inspect thread state before dispatch; return cached execution results if thread is already executed.

7. Implementation Status, Limitations, and Production Roadmap

To maintain rigorous technical transparency, DukaanPilot AI explicitly distinguishes verified implemented features from future architectural enhancements:

Existing Implemented Capabilities (Verified in 66 Automated Tests)	Proposed Future Improvements (Architecture Roadmap)
• **Structured Exception Trapping:** tools.registry.execute_action prevents runtime crashes. • **Zero Phantom Uplift:** Evaluator assigns ■0.00 revenue to failed actions. • **Durable SQLite State:** SqliteSaver preserves complete thread history across restarts. • **Transparent UI Diagnostics:** Red failure badges and error traces rendered in dashboard. • **Mandatory Re-Validation:** Custom/revised actions must re-pass schema & discount checks. • **Bilingual Voice Briefing:** Sarvam AI TTS with text chunking and SHA-256 audio caching.	• **Automatic Retries with Controlled Limits:** Jittered exponential backoff (max 3 attempts) with circuit breakers for transient I/O faults. • **Real-Time API Failure Monitoring:** OpenTelemetry tracing and Prometheus metrics for API latency and failure alarms. • **Real-World POS Tool Integrations:** Live authenticated connectors for Vyapar, Khatabook, and WhatsApp Business API. • **Compensating Rollback Sagas:** Automated reversing transactions for multi-step operational failures. • **Cryptographic Audit Trail:** Hash-chained (SHA-256) immutable ledgers for compliance and dispute resolution.

Integrity & Verification Note: *All business arithmetic, financial margins, and inventory formulas are deterministically computed with Python/Pandas. LLMs are never permitted to hallucinate business figures. All tool simulations operate safely offline. Verified against a comprehensive 66-test automated suite.*